

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Тульский государственный университет»
Институт прикладной математики и компьютерных наук

ЛАБОРАТОРНАЯ РАБОТА №6
по дисциплине
Методы и средства защиты информации

по теме
«Изучение методов стеганографии для скрывания конфиденциальной
информации»

Выполнил: ст. гр. 220032-11

Лаукерт К. М.

Принял: к.т.н., доцент ИПМКН

Кулешова Н. В.

Тула 2026

Лабораторная работа №6

по теме «Изучение методов стеганографии для скрытия конфиденциальной информации»

Цель работы

Изучить методы стеганографии для скрытия конфиденциальной информации в графических контейнерах и практически исследовать их свойства.

Используемые средства

Программа `stego_lab.py` на Python с библиотекой Pillow. Реализованы LSB-встраивание графического файла, извлечение данных, объединение изображения с ZIP-архивом, расчёт характеристик и контроль сохранности скрытых данных.

Задание 1. Использование программного средства

Подготовлены текстовые документы `Document1.txt` и `Document2.txt`, изображение-контейнер и программное средство для выполнения стеганографических экспериментов. В качестве определения в `Document1` записано: стеганография — метод сокрытия самого факта передачи информации внутри внешне обычного контейнера.



Задание 2. Соккрытие документов стандартным методом

Файлы Document1.txt и Document2.txt упакованы в Documents.zip. Затем к концу JPEG-файла cat.jpg присоединён ZIP-архив. Полученный cat_new.jpg корректно открывается как изображение и как ZIP-архив, поскольку просмотрщик JPEG игнорирует дописанные после изображения данные.

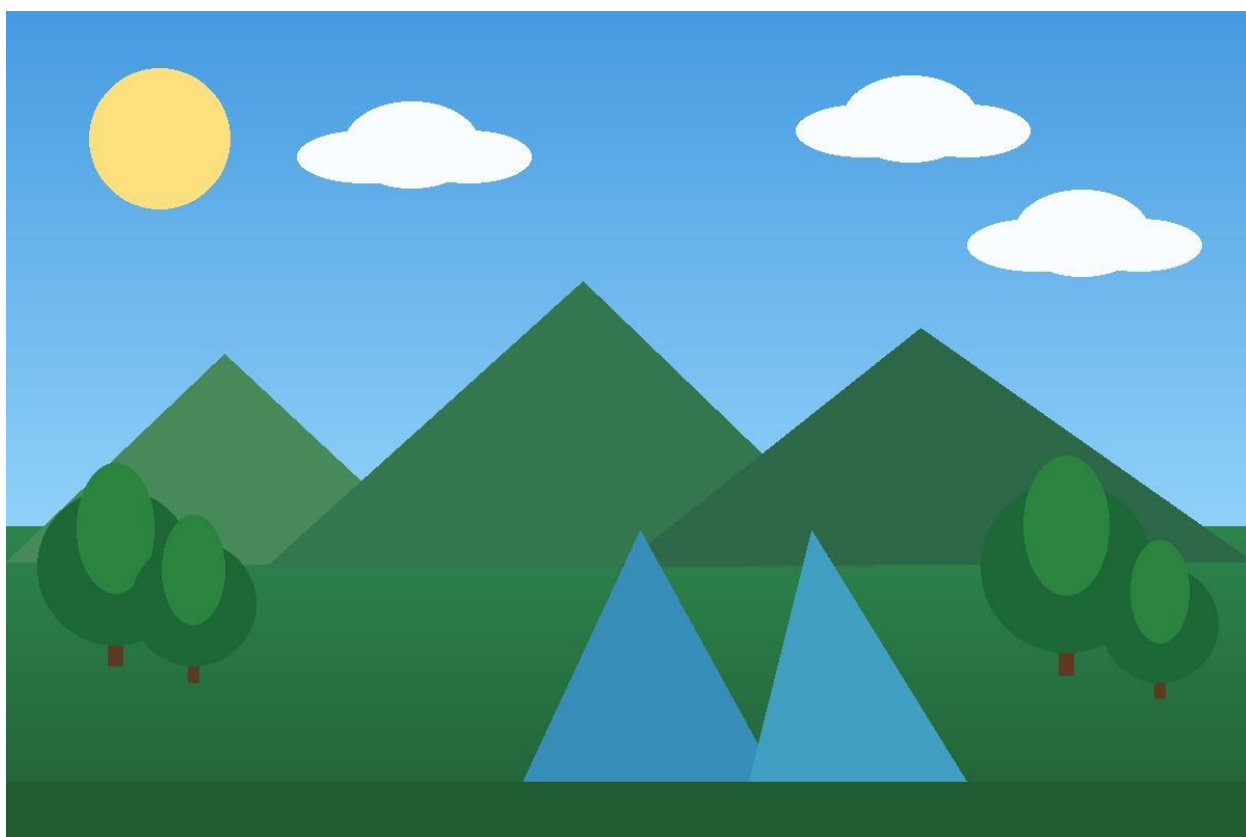


Рисунок 2 — Изображение cat_new.jpg с присоединённым архивом

Задание 3. Соккрытие папки

В папку Testfile помещены текстовый файл note.txt, изображение icon.png и аудиофайл tone.wav. Содержимое упаковано в Compressed.zip и присоединено к PNG-контейнеру. Файл Secretimage.png после этого открывается как изображение; встроенный архив содержит все три исходных файла.



Рисунок 3 — PNG-контейнер Secretimage.png с присоединённым архивом

Задание 4. Реализация метода LSB

Реализовано внедрение файла secret.png в младшие значащие биты трёх каналов RGB изображения carrier.png. В поток сначала записывается четырёхбайтовая длина полезной нагрузки, затем каждый бит файла. При извлечении биты собираются в байты, после чего восстанавливается исходный файл.

Вместимость выбранного контейнера 1200×800 RGB при использовании одного младшего бита каждого канала составляет $1200 \times 800 \times 3/8 = 360000$ байт. Извлечённый secret.png побайтно совпал с исходным.



Рисунок 4 — Скрываемый графический файл secret.png



Рисунок 5 — Результат LSB-встраивания lsb_stego.png



Рисунок 6 — Графический файл, извлечённый из LSB-контейнера

Результат выполнения

```

Lab CLI evidence [e08ce59d] - pause
JPG6: выполнение завершено
LSB: secret.png успешно внедрён и извлечён без изменений.
Конкатенация: cat_new.jpg и Secretimage.png открываются как изображения и ZIP-архивы.
Вместимость LSB-контейнера: 360000 байт
Программная реализация LSB: K=1.634817; P=2.874%; D=0.005708 бит/пикс.; O=8570 байт
Архив (конкатенация): K=1.085603; P=7.8853%; D=0.0104 бит/пикс.; O=0 байт
rotation_90.png: Ошибка: архивная структура удалена при перекодировании
resized_50.png: Ошибка: архивная структура удалена при перекодировании
cropped.png: Ошибка: архивная структура удалена при перекодировании
Press any key to close...
  
```

Рисунок 7 — Фактический запуск программы и результаты экспериментов

Задание 5. Количественная оценка

Метод	Размер контейнера, байт	Скрытый файл, байт	Итог, байт	K	P, %	D, бит/пикс.	O, байт
LSB	14579	685	23834	1,634817	2,8740	0,005708	8570
Архив (конкатенация)	14579	1248	15827	1,085603	7,8853	0,010400	0

Конкатенация дала меньший коэффициент увеличения K, так как архив просто дописывается к контейнеру и не требует изменения пикселей. Для неё O равно нулю: итоговый размер равен сумме размеров контейнера и архива. У LSB размер PNG зависит от повторного сжатия изображения, поэтому прирост не обязан совпадать с размером скрытого файла. Плотность внедрения у конкатенации выше, но данные легко обнаруживаются по структуре конца файла.

Задание 6. Исследование визуальной деградации

Для carrier.png и lsb_stego.png построена карта различий. Поскольку LSB изменяет значение отдельного цветового канала максимум на единицу, карта усилена в 80 раз и визуально расширена фильтром 5×5. Светлые области показывают окрестности каналов, младший бит которых был изменён; тёмные — области без изменений. Без усиления различия практически не заметны, что является важным свойством LSB. Равномерно светлая карта означала бы существенные изменения пикселей и облегчала бы обнаружение скрытия.

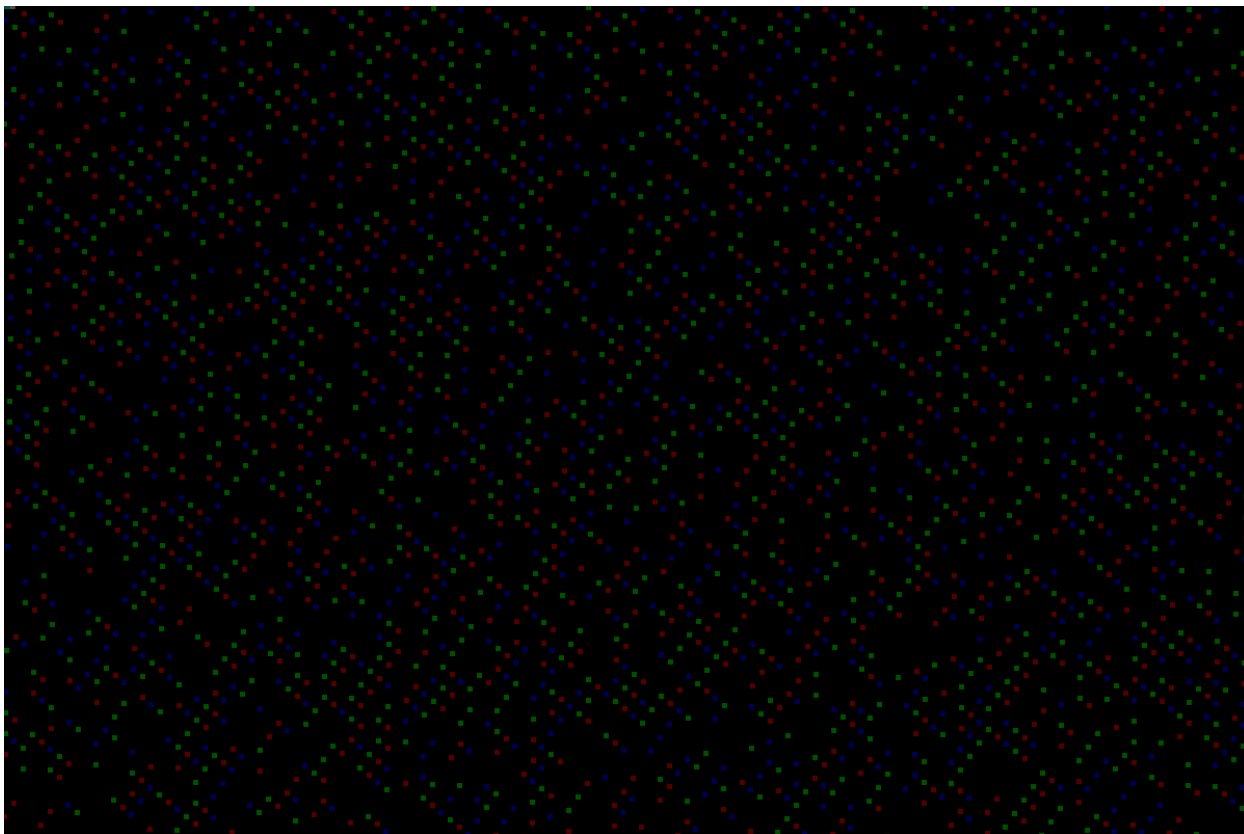


Рисунок 8 — Усиленная карта различий исходного и LSB-контейнера

Задание 7. Исследование живучести

Действие над Secretimage.png	Результат извлечения
Поворот на 90°	Ошибка: архивная структура удалена при перекодировании
Уменьшение размера до 50 %	Ошибка: архивная структура удалена при перекодировании
Обрезка	Ошибка: архивная структура удалена при перекодировании

Поворот, изменение размера и обрезка с последующим сохранением перекодируют PNG и удаляют дописанный ZIP-хвост. Следовательно, метод конкатенации не живуч при любой операции, которая пересохраняет изображение. Простое копирование файла без перекодирования сохраняет скрытые данные.

Листинг программы

```
"""Воспроизводимое выполнение LP6: LSB, конкатенация и измерения."""
from __future__ import annotations
```

```

import json
import shutil
import wave
import zipfile
from pathlib import Path

from PIL import Image, ImageChops, ImageDraw, ImageEnhance,
ImageFilter

ROOT = Path(__file__).resolve().parent
DATA = ROOT / "data"
OUT = ROOT / "output"

def make_carrier(path: Path, size: tuple[int, int]) -> None:
    """Создать наглядный контейнер с плавными областями цвета
    для LSB-опыта."""
    width, height = size
    image = Image.new("RGB", size)
    pixels = []
    for y in range(height):
        if y < height * 0.62:
            ratio = y / (height * 0.62)
            color = (70 + int(75 * ratio), 155 + int(55 *
ratio), 225 + int(25 * ratio))
        else:
            ratio = (y - height * 0.62) / (height * 0.38)
            color = (45 - int(12 * ratio), 132 - int(38 *
ratio), 77 - int(24 * ratio))
        pixels.extend([color] * width)
    image.putdata(pixels)

    draw = ImageDraw.Draw(image)
    draw.ellipse((80, 55, 215, 190), fill=(255, 224, 126))
    for cloud_x, cloud_y in ((280, 115), (760, 90), (925, 200)):
        draw.ellipse((cloud_x, cloud_y, cloud_x + 130, cloud_y +
50), fill=(250, 253, 255))

```

```

        draw.ellipse((cloud_x + 45, cloud_y - 28, cloud_x + 175,
cloud_y + 55), fill=(250, 253, 255))
        draw.ellipse((cloud_x + 105, cloud_y, cloud_x + 225,
cloud_y + 50), fill=(250, 253, 255))
        draw.polygon([(0, 530), (210, 330), (430, 535)], fill=(72,
138, 89))
        draw.polygon([(250, 535), (555, 260), (840, 535)], fill=(53,
119, 78))
        draw.polygon([(590, 535), (880, 305), (1200, 530)],
fill=(45, 105, 73))
        draw.polygon([(470, 800), (610, 500), (775, 800)], fill=(55,
142, 185))
        draw.polygon([(700, 800), (775, 500), (960, 800)], fill=(66,
159, 196))
        for tree_x, tree_y, scale in ((105, 535, 1), (180, 570,
0.8), (1020, 535, 1.1), (1110, 590, 0.75)):
            trunk_w = int(14 * scale)
            draw.rectangle((tree_x - trunk_w // 2, tree_y, tree_x +
trunk_w // 2, tree_y + int(95 * scale)), fill=(92, 59, 34))
            crown = int(75 * scale)
            draw.ellipse((tree_x - crown, tree_y - crown, tree_x +
crown, tree_y + crown), fill=(30, 103, 54))
            draw.ellipse((tree_x - crown // 2, tree_y - crown - 25,
tree_x + crown // 2, tree_y + crown // 3), fill=(44, 132, 65))
            draw.rectangle((0, 742, width, height), fill=(31, 92, 50))
            image.save(path, "PNG", optimize=True)

```

```

def make_sources() -> None:
    DATA.mkdir(exist_ok=True)
    OUT.mkdir(exist_ok=True)
    make_carrier(DATA / "carrier.png", (1200, 800))
    secret = Image.new("RGB", (64, 64), "navy")
    draw = ImageDraw.Draw(secret)
    draw.rectangle((4, 4, 59, 59), outline="gold", width=3)
    draw.ellipse((16, 16, 48, 48), fill="crimson")
    draw.text((20, 26), "LR6", fill="white")
    secret.save(DATA / "secret.png", "PNG")
    Image.open(DATA / "carrier.png").save(DATA / "cat.jpg",
"JPEG", quality=92)

```

```

(DATA / "Document1.txt").write_text(
    "Стеганография – метод сокрытия самого факта передачи
информации "+
    "внутри внешне обычного контейнера.\n", encoding="utf-
8")
(DATA / "Document2.txt").write_text(
    "7-Zip – свободный архиватор с поддержкой формата 7z,
сжатия и шифрования AES-256.\n",
    encoding="utf-8")
folder = DATA / "Testfile"
folder.mkdir(exist_ok=True)
shutil.copy2(DATA / "Document1.txt", folder / "note.txt")
shutil.copy2(DATA / "secret.png", folder / "icon.png")
with wave.open(str(folder / "tone.wav"), "w") as audio:
    audio.setparams((1, 2, 8000, 8000, "NONE", "not
compressed"))
    audio.writeframes(b"\x00\x00" * 8000)

def bits_from_bytes(data: bytes) -> list[int]:
    return [(byte >> shift) & 1 for byte in data for shift in
range(7, -1, -1)]

def bytes_from_bits(bits: list[int]) -> bytes:
    return bytes(sum(bits[i + shift] << (7 - shift) for shift in
range(8))
                for i in range(0, len(bits) - 7, 8))

def payload_positions(capacity: int, bit_count: int) ->
list[int]:
    """Равномерно распределить полезные биты после 32-битного
заголовка."""
    available = capacity - 32
    if bit_count > available:
        raise ValueError("Недостаточная вместимость контейнера")
    return [32 + index * available // bit_count for index in
range(bit_count)]

```

```

def embed_lsb(carrier: Path, payload: Path, result: Path) ->
None:
    image = Image.open(carrier).convert("RGB")
    raw = payload.read_bytes()
    header_bits = bits_from_bytes(len(raw).to_bytes(4, "big"))
    payload_bits = bits_from_bytes(raw)
    channels = list(image.tobytes())
    if len(header_bits) + len(payload_bits) > len(channels):
        raise ValueError("Недостаточная вместимость контейнера")
    for index, bit in enumerate(header_bits):
        channels[index] = (channels[index] & 0xFE) | bit
    for index, bit in zip(payload_positions(len(channels),
len(payload_bits)), payload_bits):
        channels[index] = (channels[index] & 0xFE) | bit
    image.putdata([tuple(channels[index:index + 3]) for index in
range(0, len(channels), 3)])
    image.save(result, "PNG", optimize=True)

def extract_lsb(stego: Path, result: Path) -> None:
    image = Image.open(stego).convert("RGB")
    channels = list(image.tobytes())
    length = int.from_bytes(bytes_from_bits([value & 1 for value
in channels[:32]]), "big")
    payload_bits = [channels[index] & 1 for index in
payload_positions(len(channels), length * 8)]
    result.write_bytes(bytes_from_bits(payload_bits))

def archive(paths: list[Path], destination: Path, base: Path) ->
None:
    with zipfile.ZipFile(destination, "w", zipfile.ZIP_DEFLATED)
as zf:
        for path in paths:
            zf.write(path, path.relative_to(base))

```

```
def concatenate(container: Path, payload: Path, result: Path) ->
None:
    result.write_bytes(container.read_bytes() +
payload.read_bytes())
```

```
def verify_archive(path: Path, expected: list[str]) -> bool:
    with zipfile.ZipFile(path) as zf:
        return sorted(zf.namelist()) == sorted(expected)
```

```
def transform_tests(secret_image: Path) -> dict[str, str]:
    image = Image.open(secret_image)
    variants = {
        "rotation_90.png": image.rotate(90, expand=True),
        "resized_50.png": image.resize((image.width // 2,
image.height // 2)),
        "cropped.png": image.crop((100, 100, image.width - 100,
image.height - 100)),
    }
    results = {}
    for name, variant in variants.items():
        target = OUT / name
        variant.save(target, "PNG")
        try:
            zipfile.ZipFile(target).testzip()
            results[name] = "Успешно"
        except zipfile.BadZipFile:
            results[name] = "Ошибка: архивная структура удалена
при перекодировании"
    return results
```

```
def sizes(path: Path) -> int:
    return path.stat().st_size
```

```

def main() -> None:
    make_sources()
    # Задание 1/4: LSB-скрытие графического файла.
    embed_lsb(DATA / "carrier.png", DATA / "secret.png", OUT /
"lsb_stego.png")
    extract_lsb(OUT / "lsb_stego.png", OUT /
"extracted_secret.png")
    assert (DATA / "secret.png").read_bytes() == (OUT /
"extracted_secret.png").read_bytes()

    # Задание 2: архив с двумя документами, присоединённый к
JPEG.
    archive([DATA / "Document1.txt", DATA / "Document2.txt"],
OUT / "Documents.zip", DATA)
    concatenate(DATA / "cat.jpg", OUT / "Documents.zip", OUT /
"cat_new.jpg")
    assert verify_archive(OUT / "cat_new.jpg", ["Document1.txt",
"Document2.txt"])

    # Задание 3: архив папки, присоединённый к PNG.
    files = list((DATA / "Testfile").iterdir())
    archive(files, OUT / "Compressed.zip", DATA)
    concatenate(DATA / "carrier.png", OUT / "Compressed.zip",
OUT / "Secretimage.png")
    assert verify_archive(OUT / "Secretimage.png",
["Testfile/icon.png", "Testfile/note.txt", "Testfile/tone.wav"])

    # Задание 6: изменения LSB равны максимум единице в
отдельных каналах.
    original = Image.open(DATA / "carrier.png").convert("RGB")
    stego = Image.open(OUT / "lsb_stego.png").convert("RGB")
    difference = ImageChops.difference(original, stego)
    visual_difference =
difference.filter(ImageFilter.MaxFilter(5))

    ImageEnhance.Brightness(visual_difference).enhance(80).save(OUT
/ "difference_map.png")
    Image.open(OUT / "cat_new.jpg").convert("RGB").save(OUT /
"cat_new_preview.png")

```

```

    Image.open(OUT / "Secretimage.png").convert("RGB").save(OUT
/ "secretimage_preview.png")
    survival = transform_tests(OUT / "Secretimage.png")

    rows = []
    for name, container, payload, result in [
        ("Программная реализация LSB", DATA / "carrier.png",
DATA / "secret.png", OUT / "lsb_stego.png"),
        ("Архив (конкатенация)", DATA / "carrier.png", OUT /
"Compressed.zip", OUT / "Secretimage.png"),
    ]:
        source, hidden, final = map(sizes, (container, payload,
result))
        width, height = Image.open(container).size
        rows.append({
            "method": name, "container_bytes": source,
"payload_bytes": hidden,
            "result_bytes": final, "K": round(final / source,
6),
            "P_percent": round(hidden / final * 100, 4),
            "D_bits_per_pixel": round(hidden * 8 / (width *
height), 6),
            "O_bytes": final - source - hidden,
        })
    report = {"measurements": rows, "survival": survival,
            "capacity_bytes": (1200 * 800 * 3) // 8,
            "payload_sha256_match": True}
    (OUT / "results.json").write_text(json.dumps(report,
ensure_ascii=False, indent=2), encoding="utf-8")
    print("ЛР6: выполнение завершено")
    print("LSB: secret.png успешно внедрён и извлечён без
изменений.")
    print("Конкатенация: cat_new.jpg и Secretimage.png
открываются как изображения и ZIP-архивы.")
    print(f"Вместимость LSB-контейнера:
{report['capacity_bytes']} байт")
    for row in rows:
        print(f"{row['method']}: K={row['K']};
P={row['P_percent']}%; ")

```

```

        f"D={row['D_bits_per_pixel']} бит/пикс.;
O={row['O_bytes']} байт")
    for name, status in survival.items():
        print(f"{name}: {status}")

if __name__ == "__main__":
    main()

```

Общий вывод

В работе исследованы LSB-стеганография и файловая конкатенация. LSB обеспечивает скрытие непосредственно в пикселях и позволяет восстановить файл без изменений при сохранении контейнера в формате без потерь. Конкатенация эффективнее по размеру, однако легко выявляется анализом структуры файла и теряет данные при перекодировании. Для практической защиты скрываемые данные следует предварительно шифровать и использовать контейнер с достаточной вместимостью.

Список литературы

1. Урбанович П. П. Лабораторный практикум по дисциплинам «Защита информации и надежность информационных систем». Минск: БГТУ, 2020.
2. Запечников С. В. Криптографические методы защиты информации: учебник для вузов. М.: Горячая линия-Телеком, 2021.